

ELF

Executable and Linkable Format

1. Pregled

ELF (Executable and Linkable Format) je standardni binarni format na UNIX-like sistemima. Koristi se za objektna fajlove (.o), izvršne programe i deljene biblioteke (.so). Isti format pokriva sve tri faze: kompajliranje, povezivanje (linking) i izvršavanje.

Tip fajla	Ekstenzija	Opis
relocatable	.o	Izlaz kompajtlara; sadrži masinski kod ali nedovrsene adrese. Namenjen linker-u.
executable	(bez)	Gotov program koji OS može pokrenuti. Sve adrese su razresene.
shared object	.so	Deljeni kod koji se može ulinkovati staticno ili učitati dinamički u toku izvršavanja.

2. Struktura ELF datoteke

Svaka ELF datoteka počinje obaveznim ELF headerom koji opisuje celu datoteku. Iza headera mogu slediti Program Header Table (PHT), sekcije i Section Header Table (SHT). Redosled nije fiksiran; sve lokacije su opisane ofsetima u ELF headeru.

```

+-----+
|      ELF Header      |  uvek prisutan
+-----+
| Program Header Table |  obavezan pri izvršavanju, opcioni pri linkovanju
+-----+
|   Sekcije (.text,    |
|   .data, .symtab...) |
+-----+
| Section Header Table |  obavezna pri linkovanju, opciona pri izvršavanju
+-----+

```

ELF = Executable and Linkable Format | PHT = Program Header Table | SHT = Section Header Table

ELF datoteka se može posmatrati na dva načina:

Linking view (pogled pri linkovanju) - koristi Section Header Table. Opisuje sadržaj datoteke kao kolekciju sekcija (.text, .data, .symtab, .rela.text, ...). Linker cita sekcije iz više .o fajlova, resava simbole i spaja ih u jedan izvršni program.

Execution view (pogled pri izvršavanju) - koristi Program Header Table. Opisuje sadržaj kao segmente koje OS loader preslikava u virtuelni adresni prostor procesa. Više sekcija može biti spojeno u jedan segment radi efikasnosti (npr. .text i .rodata -> RX segment).

3. Primitivni tipovi (ELF64)

ELF specifikacija definiše sopstvene tipove podataka kako bi bila nezavisna od konkretnog kompajtlara. Poravnanje je uvek jednako veličini tipa, što garantuje da strukturna polja ne zahtevaju eksplicitni padding.

Tip	Vel. (B)	Znacenje
Elf64_Addr	8	Virtuelna ili fizicka adresa
Elf64_Off	8	Offset u okviru ELF fajla
Elf64_Xword	8	Unsigned 64-bit ceo broj
Elf64_Sxword	8	Signed 64-bit ceo broj
Elf64_Word	4	Unsigned 32-bit ceo broj
Elf64_Sword	4	Signed 32-bit ceo broj
Elf64_Half	2	Unsigned 16-bit ceo broj
unsigned char	1	Unsigned 8-bit ceo broj

4. ELF Header - Elf64_Ehdr

ELF header se uvek nalazi na offsetu 0 i ima fiksnu veličinu (64 bajta za ELF64). Sadrži sve informacije potrebne da se lociraju ostale tabele i strukture unutar fajla.

```
typedef struct {
    unsigned char  e_ident[16]; // magic, class, endianness, ABI...
    Elf64_Half     e_type;       // tip fajla: ET_REL, ET_EXEC, ET_DYN
    Elf64_Half     e_machine;    // ciljna arhitektura (npr. EM_X86_64 = 62)
    Elf64_Word     e_version;    // uvek EV_CURRENT = 1
    Elf64_Addr     e_entry;      // virtuelna adresa _start(); 0 za .o fajlove
    Elf64_Off      e_phoff;      // pozicija PHT u fajlu; 0 ako PHT ne postoji
    Elf64_Off      e_shoff;      // pozicija SHT u fajlu; 0 ako SHT ne postoji
    Elf64_Word     e_flags;      // zastavice specifične za arhitekturu
    Elf64_Half     e_ehsize;     // velicina ovog headera (= 64 za ELF64)
    Elf64_Half     e_phentsize;  // velicina jednog unosa u PHT
    Elf64_Half     e_phnum;      // broj unosa u PHT
    Elf64_Half     e_shentsize;  // velicina jednog unosa u SHT
    Elf64_Half     e_shnum;      // broj unosa u SHT
    Elf64_Half     e_shstrndx;   // indeks SHT unosa za .shstrtab sekciju
} Elf64_Ehdr;
```

ET_ = ELF Type - konstante za e_type | EM_* = ELF Machine - konstante za e_machine | EI_* = ELF Ident - indeksi e_ident niza | PHT = Program Header Table | SHT = Section Header Table*

e_ident[] - identifikacioni niz

Prvih 16 bajtova svake ELF datoteke. Sadrže informacije koje su nezavisne od arhitekture i omogućavaju parsiranje ostatka fajla (da li je 32/64-bit, little/big-endian, koji OS ABI).

Bajt(i)	Konstanta	Vrednost / znacenje
0-3	EI_MAG0..MAG3	0x7f, "E", "L", "F" - magic bytes, identifikuju ELF fajl
4	EI_CLASS	ELFCLASS32=1 (32-bit ELF), ELFCLASS64=2 (64-bit ELF)
5	EI_DATA	ELFDATA2LSB=1 (little-endian), ELFDATA2MSB=2 (big-endian)

6	EI_VERSION	EV_CURRENT=1 (uvek 1, jedina definisana verzija)
7	EI_OSABI	ELFOSABI_NONE=0 (System V), ELFOSABI_LINUX=3
8	EI_ABIVERSION	Verzija ABI-ja; za OSABI_NONE uglavnom 0
9-15	EI_PAD	Nekorisceni bajtovi, moraju biti 0

ABI = Application Binary Interface - konvencija za pozive funkcija, tip podataka, sistemske pozive | OS = Operating System

e_type - tip fajla

Konstanta	Vrednost	Znacenje
ET_REL	1	Relocatable - izlaz kompajtlera, ulaz linkera
ET_EXEC	2	Executable - gotov program sa fiksnim adresama
ET_DYN	3	Shared object - .so biblioteka ili PIE izvrсни program
ET_CORE	4	Core dump - snimak stanja procesa pri padu

PIE = Position Independent Executable - izvršni program bez fiksnih adresa, učitava se na proizvoljnu adresu

e_machine - ciljna arhitektura (izbor)

Konstanta	Vrednost	Arhitektura
EM_386	3	Intel 32-bit x86
EM_ARM	40	ARM 32-bit
EM_X86_64	62	AMD/Intel 64-bit (x86-64)
EM_AARCH64	183	ARM 64-bit (AArch64)
EM_RISCV	243	RISC-V

5. Section Header Table - Elf64_Shdr

Section Header Table (SHT) je niz Elf64_Shdr struktura; svaki unos opisuje jednu sekciju. Linker koristi SHT da pronadje i interpretira sadržaj sekcija (.text, .symtab, .rela.text, ...). Lokacija SHT u fajlu: e_shoff. Broj unosa: e_shnum. Velicina jednog unosa: e_shentsize (= 64B za ELF64). Unos sa indeksom 0 (SHN_UNDEF) je rezervisan i ima sve nule.

```
typedef struct {
    Elf64_Word sh_name;      // offset u .shstrtab - string sa imenom sekcije
    Elf64_Word sh_type;      // tip sekcije (SHT_PROGBITS, SHT_SYMTAB, ...)
    Elf64_Xword sh_flags;    // bitmaska atributa (SHF_ALLOC, SHF_WRITE, ...)
    Elf64_Addr sh_addr;      // virt. adresa u memoriji; 0 ako se ne učitava
    Elf64_Off sh_offset;     // offset sekcije od pocetka fajla
    Elf64_Xword sh_size;     // velicina sekcije u bajtovima
    Elf64_Word sh_link;      // indeks "partnera" - zavisi od tipa, videti tabelu
    Elf64_Word sh_info;      // dodatni kontekst - zavisi od tipa, videti tabelu
    Elf64_Xword sh_addralign; // zahtev za poravnanje; 0 ili 1 znaci bez zahteva
    Elf64_Xword sh_entsize;  // velicina jednog zapisa ako je sekcija tabela; inace 0
} Elf64_Shdr;
```

SHT_* = Section Header Type - prefiks konstanti za sh_type | SHF_* = Section Header Flag - prefiks konstanti za sh_flags | SHN_* = Section Number - prefiks specijalnih indeksa sekcija

sh_type

Konstanta	Vred.	Opis
-----------	-------	------

SHT_NULL	0	Neaktivan unos (rezervisan indeks 0)
SHT_PROGBITS	1	Podaci definisani programom (.text, .data, .rodata)
SHT_SYMTAB	2	Tabela simbola (puna, za debugovanje i linkovanje)
SHT_STRTAB	3	Tabela stringova (null-terminated stringovi jedan za drugim)
SHT_RELA	4	Relokacioni zapisi sa eksplicitnim addend poljem (Elf64_Rela)
SHT_NOBITS	8	Sekcija bez podataka u fajlu (.bss); zauzima prostor u memoriji
SHT_REL	9	Relokacioni zapisi bez addend polja; addend je u masinskom kodu
SHT_DYNSYM	11	Redukovana tabela simbola za dinamicko povezivanje

sh_flags - bitmaska atributa sekcije

Konstanta	Znacenje
SHF_WRITE	Sekcija je upisiva tokom izvršavanja (npr. .data, .bss)
SHF_ALLOC	OS loader preslikava sekciju u memoriju (npr. .text, .data, .bss)
SHF_EXECINSTR	Sekcija sadrži izvršne instrukcije (npr. .text)

sh_link i sh_info - veze izmedju sekcija

Sekcije koje su tabele (simboli, relokacije) moraju ukazivati na pratecu sekciju. sh_link i sh_info služe za tu vezu; njihovo značenje zavisi od tipa sekcije:

sh_type	sh_link (partner sekcija)	sh_info (kontekst)
SHT_SYMTAB / SHT_DYNSYM	Indeks .strtab sekcije koja čuva imena simbola	Indeks prvog globalnog simbola (svi pre njega su lokalni)
SHT_RELA / SHT_REL	Indeks .symtab sekcije čiji simboli se relokiraju	Indeks sekcije na čiji kod se ove relokacije odnose
SHT_STRTAB / ostalo	SHN_UNDEF (0)	0

Rezervisani indeksi sekcija (sh_shndx)

Ove vrednosti se koriste u polju st_shndx tabele simbola da označe specijalne situacije umesto stvarnog indeksa sekcije:

Konstanta	Vrednost	Znacenje
SHN_UNDEF	0	Nedefinisan simbol - referenca na spoljni simbol koji jos nije resolvan
SHN_ABS	0xffff1	Apsolutna vrednost - st_value simbola se ne menja ni pri relokaciji
SHN_COMMON	0xffff2	Common simbol - neunificovani BSS objekat, velicina u st_value

BSS = Block Started by Symbol - sekcija za neinicijalizovane podatke; OS ih nuluje pri učitavanju

6. Standardne sekcije

Konvencijom definisana imena i tipovi sekcija. Linker i OS loader ih prepoznaju po imenu i tipu.

Sekcija	Tip	Atributi	Sadržaj
.text	SHT_PROGBITS	ALLOC+EXEC	Izvršne instrukcije
.data	SHT_PROGBITS	ALLOC+WRITE	Inicijalizovani globalni i staticni podaci
.bss	SHT_NOBITS	ALLOC+WRITE	Neinicijalizovani podaci; ne zauzima mesto u fajlu
.rodata	SHT_PROGBITS	ALLOC	Konstantni podaci (string literali, const varijable)
.symtab	SHT_SYMTAB	-	Tabela svih simbola (lokalni + globalni)

.strtab	SHT_STRTAB	-	Imena simbola iz .symtab (null-term. stringovi)
.shstrtab	SHT_STRTAB	-	Imena sekcija (referencira e_shstrndx)
.rela.text	SHT_RELA	-	Relokacioni zapisi za .text sekciju
.rel.text	SHT_REL	-	Relokacioni zapisi bez addend-a za .text

ALLOC = skraceno za SHF_ALLOC | EXEC = skraceno za SHF_EXECINSTR | WRITE = skraceno za SHF_WRITE

7. Tabela simbola - Elf64_Sym

Tabela simbola (.symtab) mapira imena na adrese ili vrednosti. Svaki simbol ima ime (indeks u .strtab), vrednost (adresa ili offset), velicinu i atribute. Linker koristi ovu tabelu da razresi medjufajlovske reference: kad jedan .o fajl poziva funkciju iz drugog, linker trazi njen STB_GLOBAL simbol i upisuje njenu adresu na mesto poziva.

```
typedef struct {
    Elf64_Word    st_name; // offset u .strtab - string sa imenom simbola
    unsigned char st_info; // gornja 4 bita: binding; donja 4 bita: tip
    unsigned char st_other; // vidljivost (donja 2 bita su STV_* vrednost)
    Elf64_Half    st_shndx; // indeks sekcije kojoj simbol pripada
    Elf64_Addr    st_value; // u .o fajlu: offset od pocetka sekcije
                    // u exe/so fajlu: virtuelna adresa simbola
    Elf64_Xword   st_size; // velicina simbola u bajtovima; 0 ako nije poznata
} Elf64_Sym;

// Makroi za citanje i pisanje st_info polja:
#define ELF64_ST_BIND(val)      (((unsigned char)(val)) >> 4)
#define ELF64_ST_TYPE(val)     ((val) & 0xf)
#define ELF64_ST_INFO(bind,type) (((bind) << 4) + ((type) & 0xf))
```

STB_* = Symbol Table Binding - prefiks konstanti za binding | STT_* = Symbol Table Type - prefiks konstanti za tip simbola | STV_* = Symbol Table Visibility - prefiks konstanti za vidljivost

Binding - gornja 4 bita st_info

Odredjuje vidljivost i prioritet simbola pri linkovanju:

Konstanta	Vred.	Znacenje
STB_LOCAL	0	Lokalni simbol. Vidljiv samo unutar fajla u kome je definisan. Dva razlicita .o fajla mogu imati lokalni simbol istog imena.
STB_GLOBAL	1	Globalni simbol. Vidljiv linkeru i svim ostalim .o fajlovima. Vise definicija istog simbola rezultira greskom.
STB_WEAK	2	Slabi simbol. Ako postoji i globalni simbol istog imena, globalni ima prednost. Koristi se za funkcije koje su deo biblioteka.

Tip simbola - donja 4 bita st_info

Konstanta	Vred.	Znacenje
STT_NOTYPE	0	Nespecifikovan tip (npr. simboli definisani u linker skripti)
STT_OBJECT	1	Podatak - globalna promenljiva, niz, struct
STT_FUNC	2	Funkcija ili drugi izvrsni kod
STT_SECTION	3	Sekcija - jedan takav simbol po sekciji, STB_LOCAL binding
STT_FILE	4	Ime izvornog fajla - STB_LOCAL, st_shndx = SHN_ABS

Vidljivost - st_other (donja 2 bita)

Konstanta	Vred.	Znacenje
STV_DEFAULT	0	Podrazumevano: globalni i slabi simboli su eksportovani iz .so
STV_HIDDEN	2	Simbol nije vidljiv van .so biblioteke (ne moze se dinamicki resolve-ovati)
STV_PROTECTED	3	Vidljiv spolja ali se ne moze preemptovati iz druge biblioteke

8. Tabela relokacija - Elf64_Rela / Elf64_Rel

Kad kompajler generise .o fajl, ne zna konacne adrese spoljnih simbola ni adrese sekcija. Umesto toga, upisuje privremenu vrednost (obicno 0) i kreira relokacioni zapis koji govori linkeru: "na offsetu X u sekciji Y stoji referenca na simbol Z - izracunaj pravu vrednost i upisj je". Linker prodje kroz sve relokacione zapise i zakrpi masinski kod.

```
// SHT_RELA - aditivna relokacija: addend je eksplicitno u zapisu
typedef struct {
    Elf64_Addr  r_offset; // u .o: offset u sekciji; u exe/so: virtuelna adresa
    Elf64_Xword r_info;   // gornja 32b: indeks simbola; donja 32b: tip relokacije
    Elf64_Sxword r_addend; // konstanta koja se dodaje pri izracunavanju
} Elf64_Rela;

// SHT_REL - relokacija bez addend-a: addend se citao iz masinskog koda
typedef struct {
    Elf64_Addr  r_offset;
    Elf64_Xword r_info;
} Elf64_Rel;

// Makroi za citanje r_info polja:
#define ELF64_R_SYM(i)          ((i) >> 32)          // indeks simbola
#define ELF64_R_TYPE(i)         ((i) & 0xffffffff)    // tip relokacije
#define ELF64_R_INFO(sym, type) (((Elf64_Xword)(sym)) << 32) + (type))
```

Kako linker primenjuje relokaciju

Za svaki relokacioni zapis linker izracunava "patch vrednost" po formuli specifičnoj za tip relokacije i upisuje je na adresu `r_offset`. Uobicajene velicine su oznacene formuli:

S = `st_value` simbola na koji relokacija pokazuje (iz tabele simbola)

A = addend (`r_addend` za Rela, ili vrednost vec upisana u kod za Rel)

P = adresa lokacije koja se zakrpljuje (`r_offset` nakon preslikavanja)

Primeri formula za x86-64:

`R_X86_64_PC32`: $\text{patch} = S + A - P$ (PC-relativna vrednost; relativna adresa)

`R_X86_64_32S`: $\text{patch} = S + A$ (apsolutna 32-bitna vrednost, sign-extended)

`R_X86_64_PLT32`: $\text{patch} = L + A - P$ (adresa PLT slot, za pozive deljenih funkcija)

Tipovi relokacija za x86-64 (izbor)

Konstanta	Vred.	Formula	Koristeno za
<code>R_X86_64_64</code>	1	$S + A$	Apsolutna 64-bit adresa
<code>R_X86_64_PC32</code>	2	$S + A - P$	PC-relativni poziv/skok (call, jmp) u .o fajlovima

R_X86_64_PLT32	4	L + A - P	Poziv funkcije u deljnoj biblioteci (putem PLT)
R_X86_64_32	10	S + A	Apsolutna 32-bit adresa (zero-extended)
R_X86_64_32S	11	S + A	Apsolutna 32-bit adresa (sign-extended)

PC = Program Counter - registar koji cuva adresu tekuće instrukcije (RIP na x86-64) | PLT = Procedure Linkage Table - tabela indirekcija za pozive funkcija iz deljenih biblioteka | L = adresa PLT slota za dati simbol

9. Korisni alati za inspekciju ELF fajlova

```
readelf -h file.o      # ELF header (tip, arhitektura, offseti tabela)
readelf -S file.o      # Section Header Table (sve sekcije sa atributima)
readelf -s file.o      # tabela simbola (.symtab)
readelf -r file.o      # tabela relokacija (.rel.* ili .rel.*)
readelf -l file        # Program Header Table / segmenti
objdump -d file.o      # disassembly .text sekcije
objdump -t file.o      # tabela simbola (alternativan prikaz)
nm file.o              # kratki prikaz simbola (T=text, D=data, U=undefined)
hd -v file.o | head -4 # hex dump prvih 64 bajta (ELF header)
```

Napomena: nm oznake: T/t (text), D/d (data), B/b (bss), U (undefined/extern), W/w (weak). Velika slova = globalni simboli, mala = lokalni.